CHAPTER FIVE

SYSTEM IMPLEMENTATION

5.0 Introduction

This chapter describes how the design presented in Chapter Four was turned into a working system. It explains the coding phase, including the tools, the structure of the code and examples of the implementation, then the testing of the system at the development, release and user levels, and finally the user manual for each actor.

5.1 Coding Phase

The system was coded as a single Next.js application in TypeScript, using the technologies selected in Chapter Three. Table 5.1 lists the tools used in the coding phase.

Table 5.1: Implementation Tools

Tool	Use in the Coding Phase
Next.js and React	Pages, components and server-side functions of the application
TypeScript	Programming language of the interface, backend services and AI modules
Tailwind CSS	Styling and responsive layout
Prisma ORM	Definition of the database schema and typed access to the data
MySQL	Storage of the data of the system
NextAuth and bcryptjs	Session-based authentication and hashing of passwords
Google Gemini AI service	Natural language responses of the AI assistant
Visual Studio Code, Git and GitHub	Code editing and version control

Table 5.2 describes the environment in which the system was developed and tested.

Table 5.2: Development and Test Environment

Item	Description
Operating system	Microsoft Windows 11
Runtime	Node.js version 20 with the npm package manager
Database server	MySQL server running on the development computer
Web server	Next.js development server at the local address of the computer
Browser	A current web browser, also used at a narrow width to check the responsive layout
Test data	Sample properties and one test account for each role

The code is organized by responsibility, as shown in Table 5.3. Each area of the interface has its own folder, the backend services are grouped under one folder, and the AI modules are kept in a separate library so that they can be developed and tested independently.

Table 5.3: Structure of the Source Code

Folder	Content
app/(public)	Public portal: home, properties, price prediction, AI assistant, contact
app/(auth)	Login and registration pages
app/(customer)	Customer portal: favorites, recommendations, rentals, purchases, payments
app/(dashboard)	Manager dashboard: properties, customers, rentals, inquiries, bookings
app/(admin)	Administrator dashboard: users, approvals, sales, transactions, analytics
app/api	Route Handlers: properties, requests, payments, customers, AI services
components	Reusable interface components
lib and lib/ai	Business logic and AI modules: assistant, recommendation, valuation
prisma	Database schema

The business rules of the rental and sale workflows are implemented as functions in one module, so that every Route Handler applies the same rules. Table 5.4 lists these functions.

Table 5.4: Main Functions of the Transaction Module

Function	Purpose
createRentalRequest	Creates a rental booking of a customer after checking that the property is available
createPurchaseRequest	Creates a purchase request of a customer for a property listed for sale
reviewRequest	Applies the decision of the authorized role: the manager for rentals, the administrator for sales
payTransaction	Records the payment of a transaction and issues the receipt
createDirectRental	Creates a lease directly for a customer selected by the manager
syncRentalLifecycle	Marks expired leases and returns their properties to availability

Database implementation. Each table of the data dictionary was implemented as a model in the Prisma schema, from which the MySQL tables are generated. Listing 5.1 shows the model of the FAVORITES table. The two relation lines implement the foreign keys, and the unique constraint prevents a customer from saving the same property twice.

```
model Favorite {
  id         String   @id @default(cuid())
```

```
  userId     String
  propertyId String
  createdAt  DateTime @default(now())

  user     User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  property Property @relation(fields: [propertyId], references: [id], onDelete: Cascade)

  @@unique([userId, propertyId])
  @@map("favorites")
}
```

Listing 5.1: Prisma Model of the Favorites Table

Role-based access. The role boundaries defined in Chapter Three are enforced by a middleware that runs before every protected page. Listing 5.2 shows the part that protects the administrator area: a visitor who is not logged in is sent to the login page, and a logged-in user with another role is returned to the user's own area.

```
if (isAdminRoute) {
  if (!isLoggedIn) {
    return NextResponse.redirect(new URL("/login", nextUrl));
  }
  if (role !== "ADMIN") {
    const destination = role === "CUSTOMER" ? "/customer" : "/dashboard";
    return NextResponse.redirect(new URL(destination, nextUrl));
  }
  return NextResponse.next();
}
```

Listing 5.2: Middleware Protecting the Administrator Area

AI modules. The recommendation engine scores every available property against the preferences of the Customer. Listing 5.3 shows the part of the content-based score that compares the location; the price, bedrooms, property type and area are scored in the same way. The score is then adjusted with preferences learned from the searches, favorites and views of the Customer, and each recommendation is saved with its reasons. The price prediction engine prepares the features of a property, namely city, property type, area, bedrooms, bathrooms and related attributes, and applies a trained regression model to return an estimate with a range. Ridge regression, Random Forest and Gradient Boosting models are implemented in the valuation module. The AI assistant retrieves the property records relevant to a question from the database and sends them with the question to the Gemini service.

```
// Weights: location 30, price 30, bedrooms 20, property type 10, area 10
const LOCATION_WEIGHT = 30;
if (prefs.location) {
  const prefLoc = prefs.location.toLowerCase();
  const propCity = (property.city || "").toLowerCase();
  const propLocation = (property.location || "").toLowerCase();
  if (propCity === prefLoc || propLocation.includes(prefLoc)) {
    totalScore += LOCATION_WEIGHT;
    reasons.push("Matches your preferred location");
  } else if (propCity.includes(prefLoc) || prefLoc.includes(propCity)) {
    totalScore += LOCATION_WEIGHT * 0.5;
```

```
    reasons.push("In a nearby area to your preferred location");
  }
}
```

Listing 5.3: Location Part of the Recommendation Score

Registration and passwords. When a customer registers, the system first checks that the e-mail address is not already in use and then stores the password as a hash, as shown in Listing 5.4. The original password is never saved.

```
const existing = await prisma.user.findUnique({ where: { email } });
if (existing) {
  return NextResponse.json(
    { success: false, error: "An account with this email already exists." },
    { status: 409 });
}
const hashedPassword = await bcrypt.hash(password, 12);
const user = await prisma.user.create({
  data: { name, email, password: hashedPassword, phone: phone || null, role: "CUSTOMER" },
});
```

Listing 5.4: Registration of a Customer with a Hashed Password

Completion of a sale. Listing 5.5 shows the rule that protects the sales workflow. The Administrator cannot approve a purchase until a paid payment exists for it, and when the sale is approved the status of the property is changed to Sold in the same database transaction, so the property cannot remain available by mistake.

```
const isPaid = await tx.payment.findFirst({
  where: { transaction: { purchaseRequestId: req.id }, status: "PAID" },
});
if (!isPaid) {
  throw new TxnError(
    "Cannot approve purchase until customer payment has been received.", 400);
}
// Finalize property sale
await tx.property.update({
  where: { id: property.id },
  data: { status: "SOLD", availabilityStatus: "SOLD", isActive: false },
});
```

Listing 5.5: Approval of a Sale and Update of the Property Status

Installation. The system is installed on a computer or server that has Node.js and MySQL. The address of the database and the key of the AI service are written in the environment file of the project, after which the commands in Listing 5.6 install the packages, create the tables and start the application.

```
npm install          # install the packages of the project
npm run db:push      # create the database tables from the Prisma schema
npm run dev          # start the application for development
npm run build        # build the application for production
npm run start        # start the production build
```

Listing 5.6: Commands for Installing and Starting the System

5.2 Test System Implementation

Testing checks that the system does what the requirements state and reveals defects before the system is used. Following Sommerville (2016), testing was organized in three stages: development testing, release testing and user testing.

5.2.1 Development Testing

Development testing is carried out by the developers while the system is being built. Table 5.5 shows the levels applied. The project contains automated test scripts for the authorization rules, the search and recommendation functions, the valuation module and the AI assistant, which are run against the local test environment.

Table 5.5: Levels of Development Testing

Level	What Was Tested
Unit testing	Individual functions, such as the recommendation score, the preparation of prediction features and the validation of forms
Component testing	Each Route Handler with valid and invalid input and with users of different roles
System testing	Complete workflows across the interface, services and database: property approval, rental and purchase

Evaluation of the price prediction model. The valuation module records the evaluation of the trained model. The model was trained on the 30 sample property records held in the development database, with 6 records kept aside as a test set, and was compared with three simple baseline estimates. Table 5.6 presents the recorded results. The model has a lower error than every baseline. These figures are indicative only: the sample is small and consists of sample listings with asking prices, not of collected Abudwak transaction data, so the results cannot be generalized to the real market. Ridge regression is the model used with the present sample, while the Random Forest and Gradient Boosting models are intended for use when a larger dataset is available.

Table 5.6: Recorded Evaluation of the Price Prediction Model

Method	MAE (USD)	RMSE (USD)	R^2
Baseline: overall median price	30,000	41,130	-0.25
Baseline: median by location and type	14,583	16,105	0.81
Baseline: price per square metre	45,536	54,348	-1.18
Ridge regression, test set (6 records)	6,424	7,638	0.96
Ridge regression, leave-one-out validation	8,289	15,394	0.77

5.2.2 Release Testing

Release testing checks a complete version of the system against its requirements before it is given to users. It was approached as requirements-based testing, in which each test case is derived from a functional requirement of Chapter Three. Table 5.7 presents the tests executed on the public portal in the local test environment, with the result observed for each.

Table 5.7: Release Tests Executed on the Public Portal

ID	Test Case	Expected and Observed Result	Status
RT-01	Open the home page	The home page is displayed with navigation and the property search button	Passed
RT-02	Open the properties page	Published listings are displayed with filters for city, listing type, property type, bedrooms and budget	Passed
RT-03	Open a property	The details page shows the specifications and status of the property	Passed
RT-04	Open price prediction	The form for city, type, bedrooms, bathrooms and area is displayed	Passed
RT-05	Open the AI assistant	The assistant panel opens with a welcome message and an input field	Passed
RT-06	Open the login page	The sign-in and create-account form is displayed	Passed

The role-based workflows are tested with one test account for each role. Table 5.8 lists the release test script for these workflows, with the result that the system is required to produce in each case.

Table 5.8: Release Test Script for the Role-Based Workflows

ID	Req.	Test Case	Expected Result
RT-07	FR-01	Log in as each role	Each user is redirected to the area of the user's role
RT-08	FR-01	Open another role's area	Access is refused and the user is returned to the user's own area
RT-09	FR-02, FR-03	Manager adds a property; Administrator approves it	The property becomes published and appears in the public listings
RT-10	FR-07	Customer opens recommendations	Properties are listed with a score and reasons
RT-11	FR-11	Customer submits a rental booking; Manager approves it	The rental becomes active and the property status becomes Rented
RT-12	FR-12	Manager creates a direct rental	The lease is created for the selected customer and property

ID	Req.	Test Case	Expected Result
RT-13	FR-13	Customer submits a purchase request; Administrator confirms; Customer pays; Administrator approves	The sale is completed and the property status becomes Sold
RT-14	FR-14	Open the receipt of a payment	An itemized receipt is displayed and can be printed
RT-15	FR-04	Search for a rented or sold property	The property is not offered as available

5.2.3 User Testing

User testing is the stage in which the intended users try the system in their own setting and decide whether it meets their needs. It is organized as acceptance testing with representatives of the three groups of users identified in Chapter Three: property seekers, property managers or agents, and an administrator. Each participant carries out the tasks of the participant's role from the test script in Table 5.8, for example searching for a property and submitting a rental booking, and the tester records whether each task was completed, the difficulties met and the comments of the participant. The acceptance criteria are that every task of the role can be completed without assistance from the developer and that the role boundaries are respected. The comments collected are used to correct defects and to improve the wording and layout of the interface.

Table 5.9: User Acceptance Tasks by Role

Role	Tasks Carried Out by the Participant
Property seeker	Search for a property with filters; open its details; request a price prediction; ask the AI assistant a question; register; save a favorite; submit a rental booking or a purchase request; view the receipt
Property Manager	Add a property with images and submit it for approval; register a customer; approve or reject a rental booking; create a direct rental; respond to an inquiry; confirm a viewing
Administrator	Approve or reject a property; confirm a purchase request; approve a sale after payment; suspend and reactivate an account; view the ledger, analytics and audit log

5.3 Developing User Manuals

The user manual explains how each actor uses the system. It is organized by role.

Public Visitor. (1) Open the website in a web browser. (2) Select Search Properties, choose the city, listing type, property type, bedrooms and budget, and select Apply Filters. (3) Select View Details to see the specifications, amenities and location of a property. (4) To estimate a

price, open the price prediction page, enter the city, property type, bedrooms, bathrooms and floor area, and submit the form. (5) To ask a question, select Ask AI Assistant and type the question. Figure 5.1 shows the property listing and the details page, and Figure 5.2 shows the AI assistant.

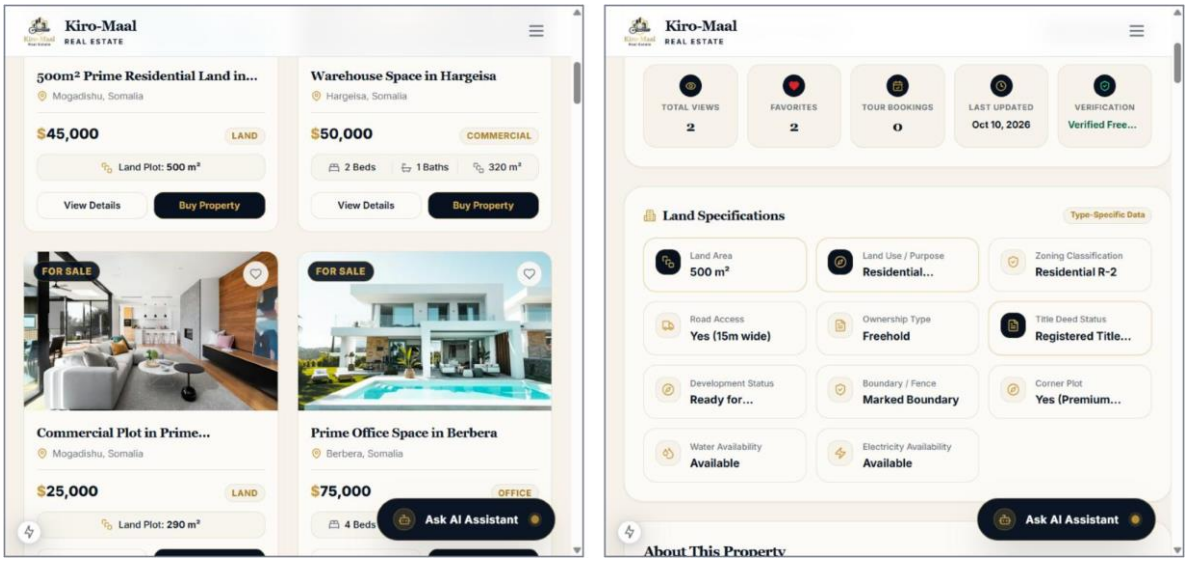


Figure 5.1: Property Listing and Property Details Page

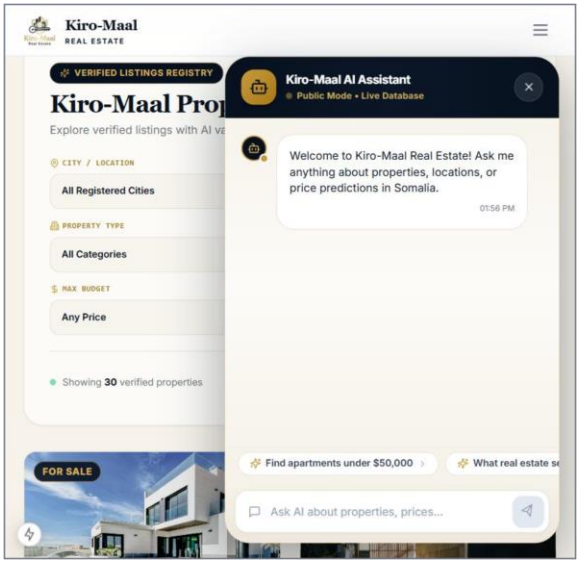


Figure 5.2: AI Assistant

Customer. (1) Select Create Account on the login page, enter the name, e-mail address and password, and sign in. (2) Select the heart icon on a property to save it as a favorite, and open Recommendations to see the properties suggested by the system. (3) To rent, open a property that is for rent, choose the rental period, complete the payment step and submit the booking; its status appears under My Rentals. (4) To buy, open a property that is for sale and submit a

purchase request; when the Administrator has confirmed it, select Pay Now under My Purchases. (5) Open Transactions to view payments and download receipts.

Property Manager. (1) Sign in to open the manager dashboard. (2) Under Properties, select Add Property, complete the steps for details, pricing, images, location and amenities, and submit the property for approval. (3) Under Customers, register or search customers. (4) Under Rentals, open a pending booking and approve or reject it, or create a rental directly by selecting the property and the customer and entering the period, rent and payment status. (5) Under Inquiries and Bookings, respond to inquiries and confirm, reschedule or cancel viewings.

Administrator. (1) Sign in to open the administrator dashboard. (2) Under Properties, open a pending property and approve it, reject it with a note, or publish it. (3) Under Requests, confirm a purchase request, and after the payment has been verified, approve the sale. (4) Under Users, create accounts, change roles, or suspend and reactivate accounts. (5) Use Transactions, Analytics and Audit Logs to monitor the activity of the platform, and Settings to configure cities and property categories.

Table 5.10 lists common problems that a user may meet and the action to take.

Table 5.10: Common Problems and Solutions

Problem	Solution
The login is refused	Check the e-mail address and password; if the account is suspended, contact the manager or administrator
A property does not appear in the public listings	The property is still pending; it appears after the Administrator approves and publishes it
The Pay Now button is not shown for a purchase	The purchase request has not yet been confirmed by the Administrator
A rental booking remains pending	The Property Manager has not yet reviewed the booking
A property cannot be rented or bought	The property is already rented or sold and is no longer available
The AI assistant does not answer	Check the internet connection and try again, because the assistant uses an external AI service

5.4 Chapter Summary

This chapter described the implementation of the system. The coding phase presented the tools, the structure of the source code and examples of the database, access control and AI implementation. Testing was organized as development, release and user testing, the recorded evaluation of the price prediction model was reported with its limitations, and the release test

cases were derived from the functional requirements. The chapter ended with the user manual for the four actors. The next chapter concludes the study.